

Complete Project Summary: Fake News Detection

For PhD Interview – May 22nd, 2026

THE ONE-LINE STORY (Say this first in interview)

"I took a published paper that claimed 99.13% accuracy, found that their results were inflated due to data leakage, fixed that problem, and built a cleaner and better system that honestly achieves 99.66% — and I also tested it on a completely different dataset and added explainability."

STEP 1 — Understanding the Original Paper

What it is (simple)

A research paper by Alotaibi et al. (2026) that tried to detect fake news using machine learning. They used a dataset of ~45,000 news articles and claimed their model was 99.13% accurate.

What they used

- Dataset: Kaggle Fake and Real News Dataset
- Method: TF-IDF (converting text to numbers) + Neural Networks (ANN, CNN, LSTM)
- Result: 99.13% accuracy

What is TF-IDF? (Definition for interview)

TF-IDF stands for Term Frequency – Inverse Document Frequency. It is a method to convert text into numbers that a machine can understand.

- **Term Frequency (TF):** How many times a word appears in one article
- **Inverse Document Frequency (IDF):** How rare that word is across ALL articles
- **Together:** Words that appear often in one article but rarely elsewhere get high scores — these are the "important" words
- **Simple example:** The word "the" appears everywhere so it gets low score. The word "manipulation" appears rarely so it gets high score in articles that use it.

How to say this in interview:

"The paper used TF-IDF for feature extraction — which is essentially a mathematical way of finding which words are important and unique to each article — and then fed those features

into neural networks for classification."

STEP 2 — Replicating the Baseline

What we did

We downloaded the exact same dataset and ran the same method. We got 99.18% accuracy — very close to their 99.13%. This confirmed our setup was correct.

Why this matters

Before improving anything, you must first prove you can reproduce the original result. This is called **baseline replication** and it is the first rule of honest research.

How to say this in interview:

"Before making any improvements, I first replicated their result to confirm I was working on the same setup. I achieved 99.18% which matched their 99.13% — so the baseline was confirmed."

STEP 3 — Discovering Data Leakage

What is Data Leakage? (Definition — very important)

Data leakage means the model accidentally "sees" test data during training. It is like giving a student the exam answers before the test. The student scores 100% but has not actually learned anything.

In machine learning there are two types:

1. **Duplicate leakage:** Same article appears in both training and test sets
2. **Feature leakage:** Information that should not be available at prediction time is included

What we found

- **6,333 duplicate articles** in the dataset (same article repeated multiple times)
- When duplicates exist, the same article can appear in BOTH training AND test sets
- This means the model memorizes that article during training and "recognizes" it in testing
- Result: Accuracy is **artificially inflated** — it looks better than it really is

Numbers

Original dataset:	44,898 articles
Duplicates found:	6,333 (14.1%)
Clean dataset:	38,565 articles

How to say this in interview:

"I discovered that the dataset contained 6,333 duplicate articles — that is 14% of the data. This means the same article could appear in both training and test sets, which is called data leakage. The model was essentially memorizing answers it had already seen, so the 99.13% accuracy was not a fair measure of real performance. This is a methodological flaw in the original paper."

Why this is PhD-level thinking:

Finding and reporting data leakage in published work shows critical evaluation skills. This is exactly what a PhD researcher is supposed to do — question existing work, not just accept it.

STEP 4 — Creating a Clean Dataset

What we did

- Removed all 6,333 duplicate articles (kept only the first occurrence)
- Did a proper stratified 80/20 split (80% for training, 20% for testing)
- Verified: **zero overlap** between training and test sets

What is Stratified Split?

When splitting data, we make sure both training and test sets have the same proportion of fake and real news. So if 55% of data is real news, both train and test sets also have 55% real news. This prevents imbalanced evaluation.

Clean dataset numbers

Clean articles:	38,565
Training set:	30,852 (80%)
Test set:	7,713 (20%)
Train-Test overlap:	0 (leakage-free)

How to say this in interview:

"After removing duplicates, I created a clean leakage-free dataset of 38,565 articles with a proper stratified split. The overlap between train and test was exactly zero — meaning the evaluation is now honest."

STEP 5 — Feature Engineering (Making Better Input)

What is Feature Engineering?

Feature engineering means creating better "descriptions" of your data for the model to learn from. Instead of just counting words, we give the model more types of information.

We used THREE types of features:

Feature Type 1: Word N-grams (TF-IDF)

- **What:** Looks at single words AND pairs of words together
- **Unigram example:** "fake", "news", "election"
- **Bigram example:** "fake news", "breaking news", "sources say"
- **Why bigrams help:** "not good" means something very different from "not" and "good" separately
- **Settings:** 3,000 features, unigrams + bigrams

Feature Type 2: Character N-grams (TF-IDF)

- **What:** Looks at sequences of 3-5 characters inside words
- **Example:** The word "president" contains char-grams like "pre", "res", "esi", "sid", "ide", "dent"
- **Why useful:** Catches writing style, spelling patterns, and even misspellings that are common in fake news
- **Settings:** 2,000 features, 3 to 5 character sequences

Feature Type 3: Meta Features (Handcrafted)

These are 7 manually calculated statistics about each article:

1. **Length** — total number of characters
2. **Word count** — number of words
3. **Caps ratio** — how much of the text is UPPERCASE (fake news often shouts)

4. **Exclamation count** — number of ! marks (fake news uses more)
5. **Question count** — number of ? marks
6. **Average word length** — longer words often appear in serious journalism
7. **Punctuation count** — commas, colons, semicolons

Why meta features? Fake news has a writing style — sensational, emotional, uses lots of caps and exclamation marks. These numbers capture that style directly.

Final Combined Matrix

Word features:	3,000
Char features:	2,000
Meta features:	7
Total:	5,007 features per article

How to say this in interview:

"Instead of just using word-level TF-IDF like the original paper, I engineered three types of features: word n-grams to capture vocabulary patterns, character n-grams to capture writing style at the letter level, and handcrafted meta features like exclamation mark count and capital letter ratio. Fake news tends to be more sensational in style, so these meta features directly capture that. The combined feature vector has 5,007 dimensions."

STEP 6 — Training Base Models

Three models trained individually:

Logistic Regression

- Simplest model — draws a straight line to separate fake from real
- Acts as a baseline within our system
- Result: **99.51% accuracy**

XGBoost (Extreme Gradient Boosting)

- Builds many small decision trees one after another
- Each tree corrects the mistakes of the previous one
- Very powerful for tabular/numerical data
- Result: **99.64% accuracy**

LightGBM (Light Gradient Boosting Machine)

- Similar to XGBoost but faster and uses less memory
- Uses a different tree-building strategy (leaf-wise instead of level-wise)
- Result: **99.64% accuracy**

How to say this in interview:

"I trained three base models — Logistic Regression as a simple baseline, XGBoost, and LightGBM, both of which are gradient boosting methods that build many decision trees sequentially. Each model alone already outperformed the original paper's 99.13% on our clean data."

STEP 7 — Stacking Ensemble (The Main Contribution)

What is a Stacking Ensemble? (Key Definition)

Stacking is a machine learning technique where you combine multiple models by:

1. Training several "base models" (Layer 1)
2. Using their predictions as INPUT to train a final "meta model" (Layer 2)

Think of it like this: three experts (LR, XGBoost, LightGBM) each give their opinion, and then a judge (meta Logistic Regression) makes the final decision based on all three opinions.

How it works in our system

Article Text



Feature Extraction (5,007 features)



Layer 1: Base Models	
LR	→ probability: 0.92
XGBoost	→ probability: 0.97
LightGBM	→ probability: 0.96



Layer 2: Meta Learner	
Logistic Regression	
Input:	[0.92, 0.97, 0.96]
Output:	FAKE or REAL

Why stacking is better than a single model

- Each model has different strengths and weaknesses
- When one model is uncertain, another might be confident
- The meta learner learns WHEN to trust each model
- Result is more robust and consistent

Our Result

Stacking Ensemble Accuracy:	99.66%
Misclassified articles:	26 out of 7,713
Original paper accuracy:	99.13% (with leakage)
Our improvement:	+0.53% on clean honest data

How to say this in interview:

"My main contribution is a two-layer stacking ensemble. In the first layer, three base models — Logistic Regression, XGBoost, and LightGBM — each produce a probability score. In the second layer, a meta Logistic Regression model takes those three probabilities as input and makes the final decision. This approach outperforms any single model and achieves 99.66% accuracy on our clean, leakage-free dataset — compared to the original paper's 99.13% which was measured on data with leakage."

STEP 8 — SHAP Explainability

What is SHAP? (Definition)

SHAP stands for SHapley Additive exPlanations. It is a method to explain WHY a model made a particular prediction.

- For each prediction, SHAP tells you which features PUSHED the prediction toward Fake and which pushed toward Real
- It comes from game theory (Shapley values) — how much did each "player" (feature) contribute to the final "outcome" (prediction)?

Why explainability matters

A model that says "this is fake" with no explanation is useless in a real newsroom. A journalist needs to know: "WHAT makes this fake?" SHAP answers that question.

What we produced

- **SHAP Bar Plot:** Shows the top 10 most important features globally across all predictions
- **SHAP Waterfall Plot:** Shows how one specific article was classified step by step
- **SHAP Force Plot:** Visual push-and-pull of features for a single prediction

How to say this in interview:

"One important gap in the original paper was that it was a black box — it classified news as fake or real but gave no reason why. I added SHAP explainability, which shows which words and features most strongly influenced each prediction. This is critical for real-world deployment in journalism, because a reporter needs to trust and understand the model's reasoning, not just its output."

STEP 9 — Cross-Dataset Test (LIAR Dataset)

What is the LIAR Dataset?

LIAR is a completely different fake news dataset from UCSB (University of California Santa Barbara). It contains 12,791 political statements labeled as true or false. It is very different from the Kaggle dataset in style, topic, and source.

Why test on a different dataset?

Testing on the same dataset you trained on — even with a clean split — only proves the model works on that specific type of data. Cross-dataset testing answers the harder question: does it generalize to different kinds of fake news?

Results

Our stacking model on Kaggle test set:	99.66%	(same domain)
DistilBERT on LIAR test set:	60.46%	(different domain)
DistilBERT trained on both datasets:	99.71% (Kaggle) / 62.43% (LIAR)	

What this tells us

The drop from 99% to 60% on LIAR is actually an HONEST and IMPORTANT finding. It means:

- Fake news detection is dataset-specific
- The Kaggle dataset has very distinct patterns (Reuters vs. political clickbait)
- Real-world fake news is much harder — it's more subtle and varied

How to say this in interview:

"To test generalization, I evaluated on the LIAR dataset — a completely different fake news benchmark. The accuracy dropped significantly to around 60%, which is an honest and important finding. It reveals that fake news detection is harder than the Kaggle benchmark suggests, because that dataset has very obvious stylistic differences between fake and real. This is a limitation I openly report and propose as future work — cross-lingual and cross-domain generalization."

STEP 10 — DistilBERT Comparison

What is DistilBERT?

DistilBERT is a smaller, faster version of BERT (Bidirectional Encoder Representations from Transformers). BERT is a massive language model trained by Google on billions of words. It understands context deeply — the word "bank" near "river" vs "bank" near "money" gets different representations.

DistilBERT keeps 97% of BERT's performance but is 40% smaller and 60% faster.

Why we used it

To show how our simpler stacking model compares against a state-of-the-art deep learning transformer model.

Results comparison

Our Stacking Ensemble:	99.66% on Kaggle	(simple, fast, explainable)
DistilBERT:	99.71% on Kaggle	(complex, slow, black box)

The key message

Our lightweight, explainable ensemble model performs almost identically to a heavy transformer model on the same data — but it is simpler, faster, and explainable. That is a strong argument for practical deployment.

How to say this in interview:

"I also compared against DistilBERT, a state-of-the-art transformer model. It achieved 99.71% on Kaggle but only 62% on LIAR. Our stacking ensemble achieved 99.66% — nearly identical performance — while being much simpler, faster, and explainable through SHAP. This supports the argument that complex transformers are not always necessary, especially when interpretability matters."

STEP 11 — Robustness Test (Noise Injection)

What is Robustness Testing?

We test whether the model still works correctly when the input text is slightly modified or corrupted. Real-world news is messy — it has typos, different capitalization, missing words.

Four types of noise we tested

- 1. **Typo noise:** Randomly change some characters (like typing errors)
- 2. **Punctuation noise:** Add or remove punctuation randomly
- 3. **Case noise:** Randomly change letters to uppercase or lowercase
- 4. **Word drop noise:** Randomly remove some words

Results at 10% noise level

Typo noise:	82.00%	(drops from 99.66%)
Punctuation noise:	76.20%	
Case noise:	99.60%	(barely affected – model lowercases everything)
Word drop noise:	95.00%	(still robust)

What this tells us

- The model is robust to case changes (because we lowercase during preprocessing)
- The model is sensitive to typos — which makes sense since TF-IDF relies on exact word matching
- This is a genuine limitation and points to future work (using character embeddings or spell correction)

How to say this in interview:

"I also did a robustness test by injecting different types of noise into the test articles. The model handles case changes perfectly because we normalize text to lowercase. However, it drops to 82% with 10% typo noise, which reveals a limitation of TF-IDF — it relies on exact word matching. This is honest evaluation and motivates future work using more robust embeddings."

COMPLETE RESULTS TABLE (For Interview Reference)

METHOD	ACCURACY	NOTES
--------	----------	-------

Original Paper (Alotaibi 2026)	99.13%	With data leakage
Our Logistic Regression	99.51%	Clean data
Our XGBoost	99.64%	Clean data
Our LightGBM	99.64%	Clean data
Our Stacking Ensemble	99.66%	Clean data ← BEST
DistilBERT (Kaggle)	99.71%	Heavy transformer
DistilBERT (LIAR)	60.46%	Cross-domain drop
Our Stacking (LIAR)	~60%	Same honest drop

HOW TO INTRODUCE THIS IN INTERVIEW

When they ask: "Tell me about your research work"

Say this (practice this paragraph):

"For my interview preparation, I took a recently published paper on fake news detection that claimed 99.13% accuracy. Instead of just reading it, I critically analyzed it and discovered a serious methodological flaw — data leakage due to 6,333 duplicate articles. This inflated their results.

I then built an improved system from scratch on the same dataset. I engineered three types of features — word n-grams, character n-grams, and handcrafted stylometric meta features — and built a two-layer stacking ensemble combining Logistic Regression, XGBoost, and LightGBM. On a clean, leakage-free dataset, my system achieved 99.66% accuracy, honestly outperforming the original paper.

I also added SHAP explainability so the model can explain WHY it flags an article as fake, and I tested generalization on the LIAR dataset which revealed the honest challenge of cross-domain fake news detection. I also compared against DistilBERT to benchmark against transformers.

This work covers the full research pipeline: critical evaluation of prior work, data cleaning, feature engineering, model design, statistical evaluation, explainability, and generalization testing."

Term	Simple Definition
TF-IDF	Math method to find important words in text
Data Leakage	Model sees test data during training — unfair
N-gram	Sequence of N words or characters together
Ensemble	Combining multiple models for better result
Stacking	Using one model's output as another's input
Meta Learner	The final model that combines base model outputs
SHAP	Method to explain which features caused a prediction
Stratified Split	Split that preserves class proportions
Robustness Test	Testing model on noisy/modified inputs
Cross-dataset	Testing on a completely different dataset
DistilBERT	Fast and smaller version of Google's BERT model
Gradient Boosting	Building trees sequentially, each fixing previous errors

LIKELY INTERVIEW QUESTIONS AND ANSWERS

Q: Why did you choose fake news detection? A: "It is a real societal problem with measurable impact. Also, the existing published benchmarks have methodological issues I wanted to address — specifically data leakage — which gave me a genuine research contribution opportunity."

Q: What is your main contribution? A: "Three things: first, I identified and documented data leakage in a published paper's benchmark. Second, I proposed a cleaner evaluation protocol and dataset. Third, I built an explainable stacking ensemble that outperforms the original honestly."

Q: Why not just use BERT? A: "DistilBERT achieved 99.71% on Kaggle — almost identical to my 99.66%. But it is much heavier, slower, and a black box. My system is lightweight and explainable through SHAP, which matters for practical deployment in newsrooms."

Q: Your accuracy drops to 60% on LIAR — isn't that bad? A: "That is actually one of the most important findings. It shows the Kaggle dataset is relatively easy because fake and real news

have very distinct styles there. Real-world fake news detection is much harder. I report this honestly as a limitation and it motivates future work on cross-domain generalization."

Q: What would you do next as PhD research? A: "Three directions: first, cross-lingual fake news detection for Hindi and regional Indian languages. Second, multimodal detection combining text with images. Third, developing a more robust model that does not degrade under noise and domain shift."

MODEL DEFINITIONS — All Models Explained Simply

Models from the ORIGINAL PAPER (Alotaibi et al.)

ANN — Artificial Neural Network

What it is: An ANN is a computing system inspired by how the human brain works. It has layers of connected "neurons" — each neuron takes some input, does a simple calculation, and passes output forward.

Structure:

Input Layer → Hidden Layer(s) → Output Layer
(text features) (learning) (Fake/Real)

How it learns:

- Each connection between neurons has a "weight" (importance)
- During training, weights are adjusted to reduce prediction errors
- This process is called backpropagation — errors flow backward and weights are updated

Two extra points:

1. **Activation functions** like ReLU introduce non-linearity — without them the network can only learn straight-line patterns, which is too simple for real text
2. **Dropout** is used during training — randomly switches off some neurons so the model does not memorize training data (prevents overfitting)

In the paper: Used Input → Dense(128, ReLU) → Dense(64, ReLU) → Output(Sigmoid). Achieved 99.13% with TF-IDF features.

Interview line: "ANN mimics the brain using layered neurons with weighted connections. It learns by repeatedly adjusting these weights to minimize prediction error through

backpropagation."

CNN — Convolutional Neural Network

What it is: CNNs were originally designed for images, but they work well for text too. Instead of looking at the whole text at once, CNN uses a "sliding window" that scans across the text looking for local patterns.

Structure:

Text Features → Conv1D Layer → Pooling → Dense → Output
(find patterns) (compress)

How it works for text:

- A filter of size 5 slides across the TF-IDF vector and detects local patterns
- Multiple filters detect different types of patterns
- Max pooling keeps only the strongest signal from each filter

Two extra points:

1. **Local feature detection** — CNN finds patterns like "sources say" or "breaking news" regardless of where they appear in the article — this is called translation invariance
2. **Hierarchical learning** — early layers detect simple patterns (single words), deeper layers combine them into complex patterns (writing style, sentence structure)

In the paper: Used Conv1D(128 filters, kernel 5) → MaxPool(2) → Conv1D(64) → GlobalMaxPool → Dense(1). Achieved 99.13% same as ANN.

Interview line: "CNN uses sliding filter windows to detect local patterns in the feature space. It is particularly good at finding short phrases and stylistic patterns regardless of their position in the text."

LSTM — Long Short-Term Memory Network

What it is: LSTM is a special type of neural network designed for sequential data — data where ORDER matters. It has a "memory cell" that can remember important information from earlier in a sequence.

Structure:

Word 1 → Word 2 → Word 3 → ... → Word N → Output

↓ ↓ ↓ ↓

(memory flows across the sequence)

The four gates:

- **Forget gate:** Decides what old information to throw away
- **Input gate:** Decides what new information to store
- **Cell state:** The actual memory that carries information forward
- **Output gate:** Decides what part of memory to use for current output

Two extra points:

1. **The key problem in this paper:** LSTM was used with TF-IDF features, which are just word count vectors with NO sequence order. So LSTM could not use its sequential memory ability — it was processing numbers, not word sequences. This is why LSTM scored lower (98.59%) than ANN/CNN despite being a more powerful architecture.
2. **Bidirectional LSTM (Bi-LSTM):** An improvement where the sequence is read both forward AND backward. A word's meaning can depend on what comes after it too. If you use proper word embeddings with Bi-LSTM, performance improves significantly.

Interview line: "LSTM is designed for sequences — it has gates that control what to remember and what to forget across time steps. However, in the original paper, LSTM was fed TF-IDF vectors which have no order information, so its sequential capability was completely wasted. That is a design flaw I identified."

Decision Tree (DT)

What it is: A Decision Tree makes predictions by asking a series of yes/no questions about the data. Like a flowchart — each question splits the data until you reach a final answer.

Example:

Does the article contain "BREAKING NEWS"?

YES → Does it have >5 exclamation marks?

YES → Likely FAKE

NO → Check next feature...

NO → Does it cite specific sources?

YES → Likely REAL

NO → Check next feature...

Two extra points:

1. **Gini Impurity** is used to decide which question to ask at each step — it measures how "mixed" the data is at each split. The model picks the question that creates the most pure (one-class) groups
2. **Weakness on high-dimensional data:** TF-IDF creates 3,000+ features. Decision trees struggle because they can only use one feature at a time per split — they cannot capture interactions between many features simultaneously. This is why DT scored only 85.67% in the paper.

Interview line: "Decision Trees use a series of if-then rules to classify data. They are interpretable but struggle with high-dimensional TF-IDF features because they cannot efficiently model complex interactions between thousands of word features."

SVM-RBF — Support Vector Machine with Radial Basis Function Kernel

What it is: SVM finds the best boundary (called a hyperplane) that separates fake and real news articles in the feature space. RBF kernel allows it to draw curved, non-linear boundaries.

How it works:

- Imagine plotting all articles as points in space
- SVM finds the line/curve that separates fake from real with the MAXIMUM gap (margin)
- Points closest to the boundary are called support vectors — only these matter for the decision

Two extra points:

1. **The kernel trick:** Real-world data is rarely linearly separable. RBF kernel mathematically projects data into a higher-dimensional space where it becomes separable — without actually computing in that high dimension. It is computationally smart.
2. **C parameter** controls the trade-off between a wider margin (more errors allowed) and a narrower margin (fewer errors). Too high C → overfitting. Too low C → underfitting. The paper used C=1.0 which is the default.

Interview line: "SVM with RBF kernel finds the optimal curved boundary between fake and real news by projecting data into a higher-dimensional space using the kernel trick. It achieved 90.22% — decent but lower than deep learning models on this high-dimensional text data."

XGBoost — Extreme Gradient Boosting

What it is: XGBoost builds many decision trees one after another. Each new tree specifically focuses on fixing the mistakes made by all previous trees. The final answer is a weighted vote of all trees.

How boosting works:

```
Tree 1: Makes some errors
Tree 2: Focuses on Tree 1's errors
Tree 3: Focuses on remaining errors
...
Tree 300: Final small corrections
→ All trees vote together → Final prediction
```

Two extra points:

1. **Regularization built-in:** XGBoost has L1 and L2 regularization terms in its objective function — this prevents individual trees from growing too complex and overfitting
2. **Why it beats neural networks on tabular data:** Our 5,007 features are structured numerical data (not raw text). XGBoost excels at this because it can capture non-linear interactions between features efficiently without needing millions of parameters

Interview line: "XGBoost is an ensemble of decision trees built sequentially where each tree corrects previous errors. It is particularly strong on structured/tabular data like our TF-IDF feature matrix, which is why it achieved 99.64% — beating the original paper's neural networks."

LightGBM — Light Gradient Boosting Machine

What it is: LightGBM is similar to XGBoost but uses a smarter tree-building strategy that makes it much faster and more memory-efficient.

Key difference from XGBoost:

- XGBoost builds trees level by level (horizontally)
- LightGBM builds trees leaf by leaf (goes deep on the most promising branch first)
- Result: LightGBM is 10-20x faster than XGBoost on large datasets

Two extra points:

1. **Histogram-based splitting:** Instead of checking every possible split point, LightGBM groups feature values into bins (histogram) and only checks those. This is why it is called "Light" — it uses much less computation

2. **GOSS (Gradient-based One-Side Sampling):** LightGBM focuses more on data points where the model is currently making large errors (high gradient) and samples from easy points — this speeds up training without losing accuracy

Interview line: "LightGBM achieves the same performance as XGBoost — 99.64% — but is significantly faster due to leaf-wise tree growth and histogram-based feature splitting. For our dataset size of 30,000+ articles, this speed advantage is practical."

Logistic Regression

What it is: Despite the name, Logistic Regression is a classification algorithm. It finds the best straight line (or hyperplane in many dimensions) that separates two classes and then converts the distance from that line into a probability using the sigmoid function.

Output:

- Gives a probability between 0 and 1
- If probability $> 0.5 \rightarrow$ Real News
- If probability $< 0.5 \rightarrow$ Fake News

Two extra points:

1. **Why it works well as a meta-learner:** When the three base models give probabilities like [0.97, 0.99, 0.98], Logistic Regression just needs to find the right weights for combining them — a simple linear combination. This is exactly what LR does best.
2. **Interpretability:** The coefficients of Logistic Regression directly tell you how much each base model's vote is trusted. This adds another layer of transparency to our stacking system.

Interview line: "Logistic Regression is a linear classifier that outputs probabilities via the sigmoid function. In our stacking system, it acts as the meta-learner — combining the three base model probabilities into a final decision. Its simplicity is a feature, not a bug, at this final combination stage."

DistilBERT

What it is: DistilBERT is a compressed version of BERT (Google's 2018 language model) created by Hugging Face. BERT reads text bidirectionally — it understands each word in context of ALL surrounding words simultaneously using an attention mechanism.

What makes it different from all above models: All the models above work on TF-IDF features — bags of words with no context. DistilBERT reads the actual sentence and understands that

"the president fired the gun" vs "the president fired the minister" have completely different meanings even though they share 4 words.

Two extra points:

1. **Pre-training advantage:** DistilBERT was trained on 3.3 billion words from Wikipedia and BookCorpus before we ever gave it our data. It already "knows" English language — we just fine-tune it for fake news specifically. This is called transfer learning.
2. **Attention mechanism:** Each word in a sentence "attends to" all other words and learns which ones matter for its meaning. This is fundamentally more powerful than TF-IDF but also much more computationally expensive — requires GPU and hours of training.

Interview line: "DistilBERT is a transformer model that reads text with full contextual understanding using attention mechanisms. Unlike TF-IDF which treats words as isolated counts, DistilBERT understands word meaning in context. However, it achieved 99.71% on Kaggle — only 0.05% better than our lightweight stacking ensemble — while being far more complex and slower."

FULL SYSTEM ARCHITECTURE — Explained in Detail

The Complete Pipeline (Step by Step)

```
RAW DATA (Fake.csv + True.csv)
    ↓
[STAGE 1: DATA PREPARATION]
    ↓
Text Cleaning
• Lowercase all text
• Remove URLs (http://...)
• Remove non-letter characters
• Remove extra spaces
    ↓
Duplicate Detection & Removal
• 6,333 duplicates found and removed
• 38,565 clean unique articles remain
    ↓
Stratified 80/20 Split
• Training: 30,852 articles
• Testing: 7,713 articles
• Overlap: 0 (verified)
    ↓
[STAGE 2: FEATURE EXTRACTION]
```

↓

Three parallel feature extractors

Word TF-IDF	Char TF-IDF	Meta
(unigrams + bigrams)	(3,4,5-char grams)	Features (7 stats)
3,000 features	2,000 features	7 features

↓

Feature Fusion (hstack)

→ Combined matrix: $30,852 \times 5,007$ (train)

→ $7,713 \times 5,007$ (test)

↓

[STAGE 3: STACKING ENSEMBLE]

↓

LAYER 1: Base Models

Logistic Regression	XGBoost	LightGBM
------------------------	---------	----------

↓

↓

↓

P(real)=0.94	P(real)=0.98	P(real)=0.97
--------------	--------------	--------------

↓

Meta-feature matrix: $[0.94, 0.98, 0.97]$

(one row per article, 3 columns = 3 model probabilities)

↓

LAYER 2: Meta Learner

Logistic Regression

Input: 3 probabilities

Output: FAKE or REAL (with confidence)

↓

[STAGE 4: EVALUATION & EXPLAINABILITY]

↓

- Accuracy, Precision, Recall, F1 on test set
- Confusion Matrix
- ROC-AUC Curve
- SHAP Feature Importance
- SHAP Waterfall Plot (single prediction)

- Cross-dataset test on LIAR
- Robustness test with noise injection

Why This Architecture Works

The architecture has three design principles:

Principle 1 — Diverse Features Each feature type captures something different. Word n-grams capture vocabulary. Character n-grams capture style. Meta features capture writing behavior. Together they give a much more complete description of an article than any single feature type alone.

Principle 2 — Diverse Models LR is linear and simple. XGBoost and LightGBM are tree-based and capture non-linear patterns. Because they make errors in different situations, their combination through stacking is more reliable than any single model.

Principle 3 — Two-Level Decision Making The meta learner does not see the raw features — it only sees HOW CONFIDENT each base model was. This means the final decision is based on consensus of experts, not raw data. If all three models are 98%+ confident, the final answer is almost certainly correct.

How to explain architecture in interview:

"My architecture has four stages. First, data preparation — cleaning text and removing 6,333 duplicates that caused leakage. Second, a three-branch feature extraction — word n-grams, character n-grams, and meta features are extracted in parallel and fused into a 5,007-dimensional vector. Third, a two-layer stacking ensemble — three base models generate probability scores which become input to a meta Logistic Regression. Fourth, evaluation — using accuracy, ROC-AUC, SHAP explainability, cross-dataset testing, and robustness testing. Each stage was designed to address a specific weakness of the original paper."

WHY LOGISTIC REGRESSION AS META LEARNER (Not XGBoost or LightGBM)

This is a very important question — interviewers love this.

The short answer:

At the meta level, the input is just 3 numbers (3 probabilities). Logistic Regression is the right tool for such simple, clean input.

Detailed reasoning:

Reason 1 — The meta input is already very informative After the base models process 5,007 features, their probability outputs already contain highly distilled information. We are not asking

the meta learner to learn complex patterns — we are asking it to combine three expert opinions. That is a simple linear task that LR handles perfectly.

Reason 2 — Overfitting risk with complex meta learners XGBoost and LightGBM are powerful because they fit complex patterns. But with only 3 input features at the meta level, a complex model would severely overfit — it would memorize the training set perfectly but fail on new data. Logistic Regression is simple enough to generalize well.

Reason 3 — What would happen if we used XGBoost as meta learner? With 3 input features and XGBoost's hundreds of trees, the model would over-engineer a solution. It might learn "if XGBoost says 0.974 exactly, answer is fake" — this is memorization not learning. LR just learns: "if average of three probabilities is high, it's real."

Reason 4 — Interpretability LR's coefficients tell you: "I trust LightGBM's vote by weight 0.8, XGBoost by 0.7, LR by 0.4." This is transparent. XGBoost as meta learner would be a black box combining black boxes — no interpretability.

Reason 5 — Speed and simplicity The meta layer runs on just 3 numbers per article. LR trains in milliseconds. Using XGBoost there would give zero performance benefit but add unnecessary complexity.

What the research literature says:

The original stacking paper by Wolpert (1992) specifically recommends using a simple, low-variance model as the meta learner to prevent overfitting at the combination stage. Logistic Regression is the most commonly used meta learner in the stacking literature for exactly this reason.

How to say this in interview:

"I chose Logistic Regression as the meta learner deliberately. After the base models process 5,007 features and output probabilities, the meta learner only receives 3 numbers. For such simple input, a complex model like XGBoost would overfit — it would memorize the training probabilities instead of learning to combine them. Logistic Regression is simple, interpretable, and perfectly suited for linearly combining three already high-quality probability estimates. This is also the standard recommendation in the stacking literature."

LIMITATIONS OF THIS WORK — Be Honest in Interview

Being able to clearly state your own work's limitations shows PhD-level maturity. Never hide weaknesses — explain them.

Limitation 1 — Dataset-Specific Patterns

What: The Kaggle dataset has a very obvious separation — all real news comes from Reuters (formal style) and all fake news is political clickbait. The model may have learned "Reuters style = real" rather than truly understanding what makes news fake.

Evidence: Accuracy drops from 99.66% to ~60% on LIAR dataset — showing the model does not generalize well to other types of fake news.

Interview answer: "The Kaggle dataset has a stylistic bias — real news is all formal Reuters journalism while fake news is sensational political content. Our model may be detecting style differences rather than factual accuracy. This is a fundamental limitation of this benchmark."

Limitation 2 — TF-IDF Cannot Understand Context

What: TF-IDF treats each word independently. It cannot understand sarcasm, irony, or context-dependent meaning. "This is not a disaster" and "This is a disaster" look almost identical to TF-IDF but mean opposite things.

Impact: Some nuanced fake news that uses formal language would fool the model. Some real news written in informal style might be misclassified.

Interview answer: "TF-IDF is a bag-of-words model — it has no understanding of word order or context. Sarcasm, irony, and subtle misinformation are invisible to it. This is why transformer models like BERT, which understand context, represent the next step for this research."

Limitation 3 — Sensitivity to Typos (Robustness)

What: Robustness testing showed accuracy drops to 82% with 10% typo noise. Since TF-IDF works on exact word matching, a single typo creates an unknown word that the model has never seen.

Impact: Real-world news often has formatting differences, abbreviations, and minor variations that could degrade performance.

Interview answer: "Our robustness tests revealed that the model is sensitive to character-level noise. A 10% typo rate drops accuracy by about 17 percentage points. This is a consequence of TF-IDF's exact-match nature. Future work should include spell correction or subword embeddings that handle misspellings gracefully."

Limitation 4 — Text Only (No Multimodal)

What: News in the real world includes images, videos, author credibility, source reputation, and social media spread patterns. Our model only uses text.

Impact: A fake news article using factual-sounding text with a misleading image would be classified as real by our system.

Interview answer: "Our system is text-only. Real-world misinformation increasingly uses multimodal manipulation — misleading images paired with partially true text. Incorporating image verification, source credibility, and social propagation patterns would significantly improve real-world applicability."

Limitation 5 — English Only

What: The entire pipeline — from TF-IDF vocabulary to SHAP explanations — works only for English. India has 22 official languages and hundreds of dialects where fake news spreads massively.

Impact: The work cannot be applied directly to Hindi, Tamil, Marathi, or other Indian language news without significant re-engineering.

Interview answer: "The system is English-only, which limits its applicability in multilingual countries like India where fake news spreads significantly in regional languages. Cross-lingual and multilingual fake news detection is an important gap that I intend to address in my PhD research."

Limitation 6 — Static Model (No Real-Time Update)

What: The model is trained once and then frozen. Language evolves — new slang, new political figures, new events create new vocabulary. A model trained in 2024 may struggle with 2026 fake news patterns.

Impact: Requires periodic retraining and a pipeline to collect new labeled data regularly.

Interview answer: "The model is static — trained once on historical data. As fake news tactics evolve and new events create new vocabulary, the model's vocabulary becomes outdated. Continual learning or periodic retraining pipelines would be needed for production deployment."

FUTURE WORK — What Comes Next

Future Direction 1 — Multilingual Fake News Detection (Most Relevant for PhD in India)

Train models for Hindi, Tamil, Bengali and other Indian languages. This requires multilingual datasets (currently very scarce), cross-lingual transfer learning (train on English, transfer to

Hindi), and possibly multilingual BERT models like mBERT or IndicBERT specifically designed for Indian languages.

Why this is a good PhD topic: India has one of the highest fake news spread rates globally, especially in regional languages during elections. The dataset gap alone is a PhD-worthy contribution.

Future Direction 2 — Multimodal Fake News Detection

Combine text analysis with image verification. Many fake news articles use real images out of context or digitally manipulated images. A multimodal system would have a text branch (our current system) + an image branch (CNN or Vision Transformer) + a fusion layer that combines both signals.

Why important: Studies show that 70% of fake news shares on social media include images. Text-only systems miss this signal entirely.

Future Direction 3 — Graph-Based Propagation Analysis

Fake news spreads in specific patterns on social media — it gets shared rapidly by accounts with specific characteristics (bots, partisan accounts). Real news spreads more gradually through verified accounts. Modeling the social graph of who shares what can be a powerful additional signal.

Approach: Use Graph Neural Networks (GNN) to model user-news interaction graphs. The paper in Related Work (CMCG model) achieved 98.53% on GossipCop using this approach.

Future Direction 4 — Real-Time Detection System

Build a production-ready API that can classify news articles in real-time as they are published. This requires model serving infrastructure, confidence thresholding (only flag when confidence > 95%), and a human-in-the-loop system for borderline cases.

Future Direction 5 — Adversarial Robustness

Fake news creators will adapt once detection systems become widespread. They will write to fool the model. Adversarial training — where the model is trained against examples specifically designed to trick it — would make the system more robust against deliberate manipulation.

How to say Future Work in interview:

"My immediate future direction is multilingual fake news detection for Indian languages — this is both academically novel and socially impactful. Medium-term, I want to add multimodal capabilities combining text and image analysis. Long-term, I am interested in graph-based propagation modeling using GNNs to leverage social network signals. Each of these is a natural PhD-sized extension of this foundational work."

ADDITIONAL INTERVIEW QUESTIONS AND ANSWERS

Q: What is the difference between precision and recall? Which is more important here? A:

"Precision means: of all articles I called fake, how many were actually fake? Recall means: of all actually fake articles, how many did I catch? In fake news detection, high recall is more important — it is worse to miss a fake article (it spreads misinformation) than to incorrectly flag a real one. However, too many false flags hurt journalist trust. Our model achieves near-perfect precision and recall both — above 99% — so the trade-off is minimal in this case."

Q: What is ROC-AUC and what did you get? A: "ROC-AUC measures how well the model separates the two classes at ALL possible thresholds — not just at 0.5. AUC of 1.0 means perfect separation. AUC of 0.5 means random guessing. Our stacking ensemble achieved ROC-AUC of 0.9997 — essentially perfect separation between fake and real news on the Kaggle test set."

Q: Why did you use 80/20 split and not cross-validation? A: "For a dataset of 38,000 articles, 80/20 is statistically sufficient — the test set has 7,713 articles which gives reliable performance estimates. Cross-validation would give more stable estimates but at much higher computational cost — especially for XGBoost and LightGBM which take significant time to train. For a paper comparing multiple models, the standard hold-out split is acceptable as long as the split is done properly — which ours is, with stratification and no leakage."

Q: How is your work different from just using scikit-learn's VotingClassifier? A:

"VotingClassifier uses hard voting (majority vote of class labels) or soft voting (average of probabilities). Stacking is more sophisticated — it **LEARNS** the optimal way to combine base model outputs rather than just averaging. The meta learner discovers that on certain types of articles, XGBoost is more reliable, while on others, LightGBM is. This learned combination is more powerful than a fixed equal-weight average."

Q: What is overfitting and how did you prevent it? A: "Overfitting means the model memorizes training data instead of learning general patterns — it performs well on training set but poorly on unseen data. I prevented overfitting through: (1) removing duplicates so the model cannot memorize repeated articles, (2) proper train-test split with zero overlap, (3) dropout in neural networks, (4) subsampling in XGBoost and LightGBM (using only 80% of data and features per tree), (5) using a simple Logistic Regression as meta learner which has low variance."

Q: The original paper used ANOVA and Tukey's HSD — why didn't you? A: "The original paper ran each model 20 times and used ANOVA to test if performance differences were statistically significant. This is good practice. I ran my models on a clean dataset with a deterministic split — I could extend this by running 20 random seeds and performing the same ANOVA analysis. This is a valid suggestion and something I would add before submitting for publication."

Q: Your SHAP was done on only 100 samples — why not the full test set? A: "SHAP with TreeExplainer computes exact Shapley values for each feature and each sample. For XGBoost with 5,007 features and 300 trees, computing SHAP for all 7,713 test samples would take significant time. 100 representative samples give statistically stable importance rankings — the global feature importance plot does not change significantly with more samples. For a final paper, I would use 500-1000 samples for more robust estimates."

Q: What would change if fake news detection needed to work in real-time? A: "For real-time deployment: first, the feature extraction pipeline (TF-IDF transformation) must be pre-loaded in memory — not re-fitted. Second, inference time for our stacking model is milliseconds per article — suitable for real-time. Third, a confidence threshold should be set — only flag articles where model confidence exceeds 95%. Borderline cases (confidence 60-95%) should go to human reviewers. Fourth, the model should be retrained monthly with new data to keep up with evolving language."

Q: Did you check if subject column leaked into features? A: "Yes, this was my first sanity check when LightGBM showed suspiciously high accuracy. I checked that the clean_text column only contained the article body text — not the subject tag. The subject column in the Kaggle dataset (like 'politicsNews' vs 'worldnews') is a perfect separator of fake vs real, so including it would be a major leakage. Our clean_text comes only from the text column after URL and punctuation removal — no subject information is included."

Q: How would you handle class imbalance if it existed? A: "Our dataset is roughly balanced — 55% real, 45% fake — so class imbalance is not a major concern here. If imbalance were present (say 90% real, 10% fake), I would use: SMOTE (synthetic minority oversampling) to create artificial fake news examples, class-weight parameter in models to penalize errors on the minority class more heavily, and evaluate using F1-score and PR-AUC instead of accuracy which is misleading for imbalanced data."

Q: You have a B.Tech in IT — how does this project connect to your PhD plans? A: "This project demonstrates I can independently identify a research gap, build a complete experimental pipeline, and produce honest, reproducible results — which are the core PhD skills. Specifically, it connects to my interest in NLP, misinformation detection, and AI for social good. I am interested in extending this to multilingual settings relevant to India, which would be a novel and impactful PhD contribution combining my IT background with NLP research."

Q: What is the F1 score and why use it alongside accuracy? A: "F1 score is the harmonic mean of precision and recall. Accuracy can be misleading — if 90% of articles are real, a model that always says real gets 90% accuracy while being useless. F1 penalizes models that sacrifice either precision or recall. Our model achieves F1 above 0.99 for both classes — confirming that the high accuracy is not just due to class distribution but reflects genuine classification ability."

Q: What is the confusion matrix and what did yours show? A: "A confusion matrix shows four numbers: True Positives (correctly called real), True Negatives (correctly called fake), False Positives (real wrongly called fake), False Negatives (fake wrongly called real). Ours showed only 26 misclassifications out of 7,713 — most of those were False Negatives (fake articles the model missed), which is the more dangerous error type in our application."

Q: How is stacking different from bagging and boosting? A: "Bagging trains multiple models on random subsamples and averages their outputs — Random Forest is the classic example. Boosting trains models sequentially where each one corrects previous errors — XGBoost and LightGBM do this. Stacking is different: it trains diverse models independently and then trains a SECOND model to optimally COMBINE their outputs. The key difference is that the combination weights are learned, not fixed. In bagging and boosting, all base models are the same type. In stacking, they can be completely different algorithms."

WHAT ELSE THE PAPER SHOULD CONTAIN (Checklist)

For when you write this up formally:

1. **Abstract** — 250 words summarizing problem, method, results, contribution
2. **Introduction** — Fake news problem, why it matters, gap in existing work (leakage), your contribution
3. **Related Work** — Cite 10-15 recent papers including Alotaibi et al., BERT papers, stacking papers
4. **Dataset Description** — Kaggle dataset stats, duplicate analysis, leakage discussion with table
5. **Methodology** — Feature engineering details, model architectures, stacking design, why these choices
6. **Experimental Setup** — Hardware specs, hyperparameters, train/test split details, evaluation metrics
7. **Results** — Table comparing all models, confusion matrices, ROC curves
8. **SHAP Analysis** — Feature importance plots, interpretation of top features
9. **Cross-Dataset Evaluation** — LIAR results, honest discussion of generalization gap
10. **Robustness Analysis** — Noise injection results, what they reveal
11. **Limitations** — Data bias, TF-IDF constraints, English-only, static model
12. **Future Work** — Multilingual, multimodal, adversarial robustness
13. **Conclusion** — Summary of contributions, impact statement

Key figures for the paper:

- Figure 1: System architecture diagram (you already have this)
- Figure 2: Class distribution before/after deduplication
- Figure 3: Duplicate analysis bar chart
- Figure 4: Meta feature distributions
- Figure 5: Model comparison bar chart
- Figure 6: Confusion matrix of stacking ensemble
- Figure 7: ROC-AUC curve
- Figure 8: SHAP summary plot
- Figure 9: SHAP waterfall plot (single prediction)
- Figure 10: Robustness test results (noise vs accuracy)